

Exercice : Un bras articulé à trois segments

Consigne

Construire un bras articulé à trois segments : une épaule à l'origine, un coude à un bras de distance, une main à un avant-bras du coude. Chaque articulation porte sa pose dans le repère de la précédente, et la pose d'une articulation dans le monde s'obtient en composant celle de son parent avec la sienne. Afficher la position du coude et celle de la main dans l'espace du monde. Faire tourner l'épaule et vérifier que la main suit.

Principe

Chaque articulation est une pose locale, exprimée dans le repère de son parent. La pose dans le monde s'obtient en composant :

$$P_{\text{monde}}(\text{enfant}) = P_{\text{monde}}(\text{parent}) \circ P_{\text{locale}}(\text{enfant})$$

L'ordre est toujours le même : le parent d'abord, l'enfant ensuite.

Programme complet

```
#include <array>
#include <cstdio>
#include <cmath>

struct Vec3 { float x, y, z; };
struct Quat { float w, x, y, z; };
struct Pose { Vec3 position; Quat orientation; };

// --- Quaternion : rotation d'un vecteur ---
Vec3 Rotation(const Quat& q, const Vec3& v)
{
    Vec3 u = { q.x, q.y, q.z };
    Vec3 uv = {
        u.y * v.z - u.z * v.y,
        u.z * v.x - u.x * v.z,
        u.x * v.y - u.y * v.x
    };
    Vec3 t = {
        u.y * uv.z - u.z * uv.y + q.w * v.x,
        u.z * uv.x - u.x * uv.z + q.w * v.y,
        u.x * uv.y - u.y * uv.x + q.w * v.z
    };
    return {
        v.x + 2.0f * t.x,
        v.y + 2.0f * t.y,
        v.z + 2.0f * t.z
    };
}

// --- Quaternion : produit ---
Quat Produit(const Quat& a, const Quat& b)
{
    return {
        a.w * b.w - a.x * b.x - a.y * b.y - a.z * b.z,
        a.w * b.x + a.x * b.w + a.y * b.z - a.z * b.y,
        a.w * b.y - a.x * b.z + a.y * b.w + a.z * b.x,
        a.w * b.z + a.x * b.y - a.y * b.x + a.z * b.w
    };
}
```

```

    };
}

// --- Pose : application a un point ---
Vec3 Appliquer(const Pose& pose, const Vec3& p)
{
    Vec3 tourne = Rotation(pose.orientation, p);
    return {
        tourne.x + pose.position.x,
        tourne.y + pose.position.y,
        tourne.z + pose.position.z
    };
}

// --- Pose : composition (parent puis enfant) ---
Pose Composer(const Pose& parent, const Pose& enfant)
{
    Quat q = Produit(parent.orientation, enfant.orientation);
    Vec3 tp = Rotation(parent.orientation, enfant.position);
    Vec3 t = {
        parent.position.x + tp.x,
        parent.position.y + tp.y,
        parent.position.z + tp.z
    };
    return { t, q };
}

// --- Quaternion : rotation autour de l'axe Y ---
Quat RotationY(float angle)
{
    float h = angle * 0.5f;
    return { std::cos(h), 0.0f, std::sin(h), 0.0f };
}

int main()
{
    // Longueurs des segments (en metres).
    const float bras = 0.30f; // epaule -> coude
    const float avantBras = 0.25f; // coude -> main

    // --- Poses locales ---

    // Epaule : a l'origine, tournee de 45 degres autour de Y.
    Pose epaule;
    epaule.position = { 0.0f, 1.50f, 0.0f }; // hauteur d'epaule
    epaule.orientation = RotationY(3.14159f / 4.0f);

    // Coude : a un bras de distance, le long de l'axe X local.
    Pose coude;
    coude.position = { bras, 0.0f, 0.0f };
    coude.orientation = RotationY(0.0f);

    // Main : a un avant-bras du coude, le long de l'axe X local.
    Pose main;
    main.position = { avantBras, 0.0f, 0.0f };
    main.orientation = RotationY(0.0f);

    // --- Poses dans le monde ---
    Pose epauleMonde = epaule;

```

```

Pose coudeMonde = Composer(epauleMonde, coude);
Pose mainMonde  = Composer(coudeMonde, main);

// --- Affichage ---
std::printf("Position du coude : %.4f %.4f %.4f\n",
            coudeMonde.position.x,
            coudeMonde.position.y,
            coudeMonde.position.z);

std::printf("Position de la main : %.4f %.4f %.4f\n",
            mainMonde.position.x,
            mainMonde.position.y,
            mainMonde.position.z);

// --- Faire tourner l'épaule ---
std::printf("\n--- Rotation de l'épaule de 90 degres ---\n");

epaule.orientation = RotationY(3.14159f / 2.0f);
epauleMonde = epaule;
coudeMonde  = Composer(epauleMonde, coude);
mainMonde   = Composer(coudeMonde, main);

std::printf("Position du coude : %.4f %.4f %.4f\n",
            coudeMonde.position.x,
            coudeMonde.position.y,
            coudeMonde.position.z);

std::printf("Position de la main : %.4f %.4f %.4f\n",
            mainMonde.position.x,
            mainMonde.position.y,
            mainMonde.position.z);

return 0;
}

```

Exemple de sortie

```

Position du coude : 0.2121 1.5000 0.2121
Position de la main : 0.3889 1.5000 0.3889

```

```

--- Rotation de l'épaule de 90 degres ---
Position du coude : 0.0000 1.5000 0.3000
Position de la main : 0.0000 1.5000 0.5500

```

Vérification

Avant rotation (épaule à 45). L'épaule est à $(0, 1.5, 0)$, tournée de 45 autour de y . Le coude est à 0.30 le long de l'axe x local. Une rotation de 45 autour de y transforme $(0.30, 0, 0)$ en :

$$(0.30 \cos 45, 0, -0.30 \sin 45) = (0.2121, 0, -0.2121)$$

Puis on ajoute la position de l'épaule $(0, 1.5, 0)$:

$$\text{coude} = (0.2121, 1.5, -0.2121)$$

Après rotation (épaule à 90). La rotation de 90 autour de y transforme $(0.30, 0, 0)$ en $(0, 0, -0.30)$. Puis on ajoute $(0, 1.5, 0)$:

$$\text{coude} = (0, 1.5, -0.30)$$

La main suit : elle est à 0.25 du coude le long de l'axe x local. Le coude ayant tourné avec l'épaule, la main tourne aussi. Sa position passe de $(0.3889, 1.5, -0.3889)$ à $(0, 1.5, -0.55)$.

Pourquoi la main suit

La pose de la main dans le monde est obtenue en composant :

$$P_{\text{main}} = P_{\text{épaule}} \circ P_{\text{coude}} \circ P_{\text{main}}$$

Si l'épaule tourne, $P_{\text{épaule}}$ change. La composition change donc aussi, et la main suit sans qu'on ait rien à lui dire. C'est exactement le principe décrit dans le chapitre : « Faites tourner l'épaule, et la main suit sans qu'on ait rien à lui dire. »

À retenir

L'ordre de composition est toujours le même : le parent d'abord, l'enfant ensuite. La pose d'une articulation dans le monde s'obtient en composant celle de son parent avec la sienne. C'est ce qui permet de construire des chaînes articulées (bras, mains, doigts) sans avoir à recalculer chaque position à la main.